**Name: Srikar Putta** **ID NO:2520080038**

**Week 12**

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
 GNU nano 8.7.1                                    src/large_pipeline.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define JOBS 4

int main() {
    pid_t pids[JOBS];
    int status;
    time_t start, end;

    printf("Starting %d pipeline jobs...\n", JOBS);

    start = time(NULL);

    for (int i = 0; i < JOBS; i++) {
        pids[i] = fork();

        if (pids[i] < 0) {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0) {
            printf("Job %d started. PID = %d\n", i + 1, getpid());

            sleep(i + 1);

            printf("Job %d completed. PID = %d\n",
                   i + 1, getpid());

            return 0;
        }
    }

    for (int i = 0; i < JOBS; i++) {
        waitpid(pids[i], &status, 0);

        if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
            printf("Job %d: SUCCESS\n", i + 1);
        else
            printf("Job %d: FAILED\n", i + 1);
    }

    end = time(NULL);

    printf("\nAll jobs completed.\n");
    printf("Total execution time: %ld seconds\n",
           end - start);

    printf("Pipeline stability check: PASSED\n");

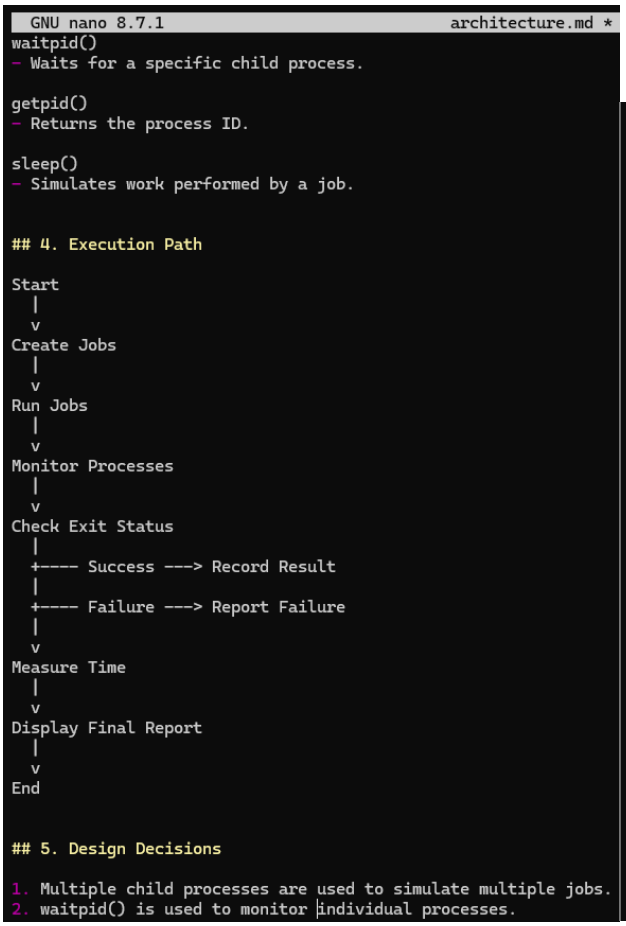
    return 0;
}
```

OUTPUT:

```
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ gcc src/large_pipeline.c -o bin/large_pipeline
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ ./bin/large_pipeline
Starting 4 pipeline jobs...
Job 1 started. PID = 2535
Job 2 started. PID = 2536
Job 3 started. PID = 2537
Job 4 started. PID = 2538
Job 1 completed. PID = 2535
Job 1: SUCCESS
Job 2 completed. PID = 2536
Job 2: SUCCESS
Job 3 completed. PID = 2537
Job 3: SUCCESS
Job 4 completed. PID = 2538
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
```

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation



```
 GNU nano 8.7.1                                  architecture.md *
waitpid()
- Waits for a specific child process.

getpid()
- Returns the process ID.

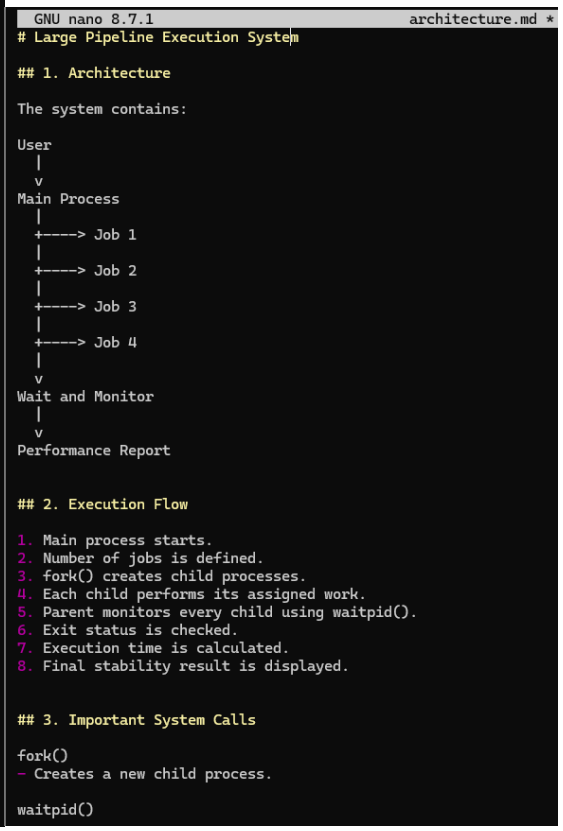
sleep()
- Simulates work performed by a job.


## 4. Execution Path

Start
  |
  v
Create Jobs
  |
  v
Run Jobs
  |
  v
Monitor Processes
  |
  v
Check Exit Status
  |
  +---- Success ---> Record Result
  |
  +---- Failure ---> Report Failure
  |
  v
Measure Time
  |
  v
Display Final Report
  |
  v
End


## 5. Design Decisions

1. Multiple child processes are used to simulate multiple jobs.
2. waitpid() is used to monitor individual processes.
```



```
 GNU nano 8.7.1                                  architecture.md *
# Large Pipeline Execution System

## 1. Architecture

The system contains:

User
  |
  v
Main Process
  |
  +----> Job 1
  |
  +----> Job 2
  |
  +----> Job 3
  |
  +----> Job 4
  |
  v
Wait and Monitor
  |
  v
Performance Report

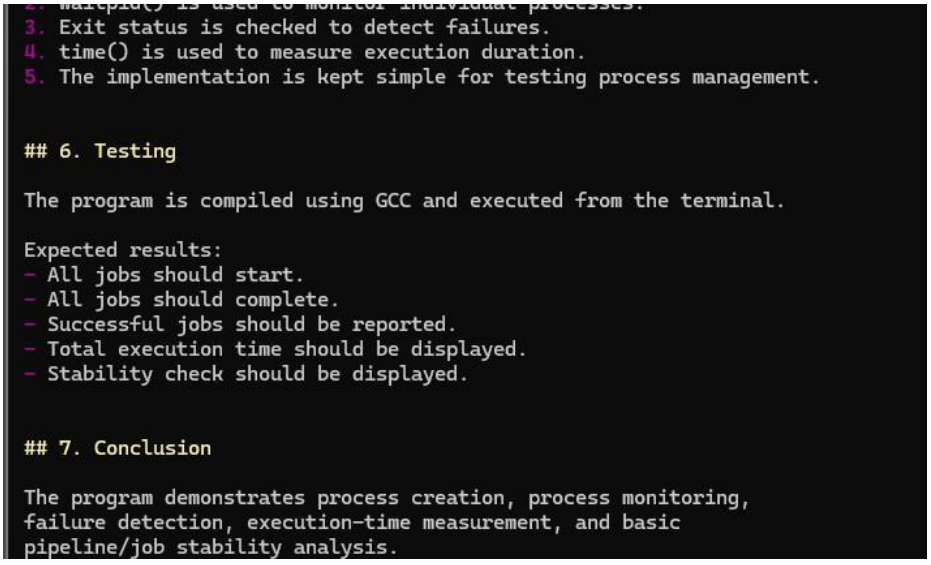

## 2. Execution Flow

1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
7. Execution time is calculated.
8. Final stability result is displayed.


## 3. Important System Calls

fork()
- Creates a new child process.

waitpid()
```



```
2. waitpid() is used to monitor individual processes.
3. Exit status is checked to detect failures.
4. time() is used to measure execution duration.
5. The implementation is kept simple for testing process management.


## 6. Testing

The program is compiled using GCC and executed from the terminal.

Expected results:
- All jobs should start.
- All jobs should complete.
- Successful jobs should be reported.
- Total execution time should be displayed.
- Stability check should be displayed.


## 7. Conclusion

The program demonstrates process creation, process monitoring,
failure detection, execution-time measurement, and basic
pipeline/job stability analysis.
```

Output:

```
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ nano architecture.md
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ cat architecture.md
# Large Pipeline Execution System

## 1. Architecture

The system contains:

User
  |
  v
Main Process
  |
  +----> Job 1
  |
  +----> Job 2
  |
  +----> Job 3
  |
  +----> Job 4
  |
  v
Wait and Monitor
  |
  v
Performance Report


## 2. Execution Flow

1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
7. Execution time is calculated.
8. Final stability result is displayed.
```

3. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
 GNU nano 8.7.1                                Makefile *
CC = gcc
CFLAGS = -Wall
TARGET = bin/large_pipeline
SOURCE = src/large_pipeline.c

all:
        $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)

run: all
        ./$(TARGET)

clean:
        rm -f $(TARGET)

check:
        @echo "Checking project structure..."
        @test -f src/large_pipeline.c && echo "Source file: OK" || echo "Source file: MISSING"
        @test -f architecture.md && echo "Documentation: OK" || echo "Documentation: MISSING"
        @test -d src && echo "src/: OK" || echo "src/: MISSING"
        @test -d bin && echo "bin/: OK" || echo "bin/: MISSING"
```

Output:

```
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ nano Makefile
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ make
gcc -Wall src/large_pipeline.c -o bin/large_pipeline
nocturnal@nocturnal:~$ /OS_Skill/Skill-Week-12$ make run
gcc -Wall src/large_pipeline.c -o bin/large_pipeline
./bin/large_pipeline
Starting 4 pipeline jobs...
Job 1 started. PID = 2663
Job 2 started. PID = 2664
Job 3 started. PID = 2665
Job 4 started. PID = 2666
Job 1 completed. PID = 2663
Job 1: SUCCESS
Job 2 completed. PID = 2664
Job 2: SUCCESS
Job 3 completed. PID = 2665
Job 3: SUCCESS
Job 4 completed. PID = 2666
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
nocturnal@nocturnal:~$ make check
Checking project structure...
Source file: OK
Documentation: OK
src/: OK
bin/: OK
```

4. To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
 GNU nano 8.7.1                                README.md *
make check

## Clean Build

make clean

## System Calls Used

- fork()
- waitpid()
- getpid()
- sleep()

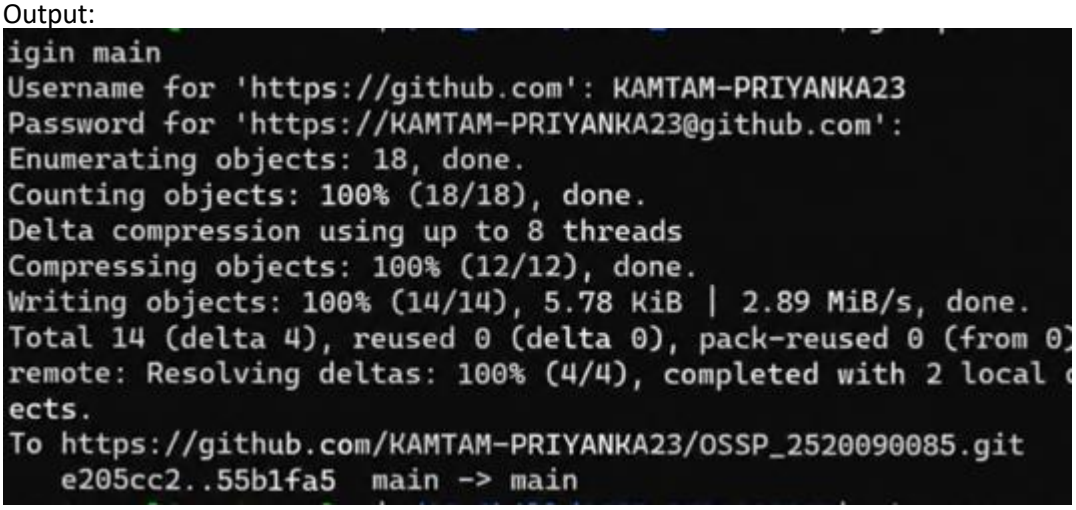
## Testing

The project creates multiple child processes, monitors their
execution, checks their exit status, and measures execution time.

## Conclusion

The project demonstrates process creation, monitoring,
performance measurement, documentation, build automation,
and final project validation.
```

Output:


igin main
Username for 'https://github.com': KAMTAM-PRIYANKA23
Password for 'https://KAMTAM-PRIYANKA23@github.com':
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 5.78 KiB | 2.89 MiB/s, done.
Total 14 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local
ects.
To https://github.com/KAMTAM-PRIYANKA23/OSSP_2520090085.git
e205cc2..55b1fa5 main -> main